

Metadatos ICFES

icfes: competencia: - formulacion_ejecucion nivel_dificultad: 3 contenido: categoria: geometria tipo: no_generico contexto: matematico eje_axial: eje2 componente: geometrico_metrico

```
options(OutDec = ".") # Asegurar punto decimal en este chunk

# Establecer semilla aleatoria
set.seed(sample(1:10000, 1))

# Aleatorizamos los nombres para contextualizar el problema
nombres <- c("Camilo", "Andrés", "Sofía", "Manuel", "Laura", "Carlos",
             "Daniela", "Miguel", "Valentina", "Eduardo", "Natalia",
             "José", "Isabella", "Gabriel", "Mariana", "Santiago", "Lucía",
             "Alejandro", "Catalina", "Mateo", "Valeria", "Sebastián", "Juliana")
nombre <- sample(nombres, 1)

# Aleatorizamos los tipos de recipientes
recipientes <- c("cilindro", "tubo", "conducto", "tanque cilíndrico",
                "recipiente cilíndrico", "contenedor cilíndrico",
                "ducto cilíndrico", "depósito cilíndrico", "cañería cilíndrica")
recipiente <- sample(recipientes, 1)

# Aleatorizar adjetivos para el cilindro interno
adjetivos_interno <- c("interno", "hueco", "vacío", "interior", "central", "medio")
adjetivo_interno <- sample(adjetivos_interno, 1)

# Aleatorizar líquidos
liquidos <- c("aceite", "combustible", "líquido", "fluido", "agua", "refrigerante",
             "solución", "mezcla", "sustancia")
liquido <- sample(liquidos, 1)

# Aleatorizar lo que se desea calcular
calculos <- c("la cantidad de", "el volumen de", "cuánto", "qué cantidad de",
             "cuántos litros de", "qué volumen de", "la capacidad de")
calculo <- sample(calculos, 1)

# Aleatorizar verbos para la acción
verbos <- c("llenar", "contener", "almacenar", "ocupar", "requerir")
verbo <- sample(verbos, 1)

# Aleatorizar medidas del cilindro (manteniendo consistencia matemática)
# Primero generamos el radio interno (entre 0.1 y 0.4 metros)
r_int <- round(runif(1, 0.1, 0.4), 2)

# Luego generamos el radio externo, asegurando que sea mayor que el interno
# Radio externo entre 1.5 y 2.5 veces el radio interno
factor_radio <- runif(1, 1.5, 2.5)
r_ext <- round(r_int * factor_radio, 2)

# Calculamos el diámetro externo (exactamente 2 veces el radio externo)
d_ext <- 2 * r_ext

# Altura del cilindro (entre 0.5 y 4 metros)
```

```

altura <- round(runif(1, 0.5, 4), 2)

# Calcular el grosor de la pared del cilindro
grosor <- round(r_ext - r_int, 1) # Asegurar que grosor se redondea a 1 decimal

# Verificar coherencia física
if (grosor < 0.05) {
  # Asegurar un grosor mínimo de 0.05 unidades
  grosor <- round(runif(1, 0.05, 0.2), 2)
  r_ext <- r_int + grosor
  # Asegurar que el diámetro externo sea exactamente 2 veces el radio externo
  d_ext <- 2 * r_ext
}

# Calcular el volumen necesario (es el volumen del cilindro interior)
volumen <- round(pi * (r_int^2) * altura, 2)

# Aleatorizar unidades de medida
unidades_longitud <- c("m", "cm", "dm")
unidad <- sample(unidades_longitud, 1)

# Ajustar valores según la unidad
factor_conversion <- 1
if (unidad == "cm") {
  factor_conversion <- 100
  d_ext <- d_ext * factor_conversion
  r_int <- r_int * factor_conversion
  altura <- altura * factor_conversion
  volumen <- volumen * (factor_conversion^3)
  r_ext <- r_ext * factor_conversion
  grosor <- grosor * factor_conversion
} else if (unidad == "dm") {
  factor_conversion <- 10
  d_ext <- d_ext * factor_conversion
  r_int <- r_int * factor_conversion
  altura <- altura * factor_conversion
  volumen <- volumen * (factor_conversion^3)
  r_ext <- r_ext * factor_conversion
  grosor <- grosor * factor_conversion
}

# Redondear todos los valores para mayor claridad
d_ext <- round(d_ext, 1)
r_int <- round(r_int, 1)
altura <- round(altura, 1)
r_ext <- round(r_ext, 1)
grosor <- round(grosor, 1)
volumen <- round(volumen, 1)

# La respuesta correcta es "C) Altura del cilindro"
# Ya que para calcular el volumen necesitaríamos conocer la altura
opcion_correcta <- 3 # Índice de "Altura del cilindro" en el vector de opciones

```

```

# Vector de solución para r-exams (1 para la correcta, 0 para las incorrectas)
solucion <- c(0, 0, 1, 0)

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Función para formatear valores numéricos (1 decimal para decimales, 0 para enteros)
formatear_numero <- function(numero) {
  if (numero %% 1 == 0) {
    return(as.integer(numero))
  } else {
    return(sprintf("%.1f", numero))
  }
}

# Formatear los valores
d_ext_fmt <- formatear_numero(d_ext)
r_ext_fmt <- formatear_numero(r_ext)
r_int_fmt <- formatear_numero(r_int)
altura_fmt <- formatear_numero(altura)
grosor_fmt <- formatear_numero(grosor) # Asegura consistencia usando el grosor precalculado

# Código Python para generar el cilindro hueco similar a la imagen de referencia
codigo_python <- paste0("
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import Ellipse, FancyArrowPatch, Polygon

# Parámetros del cilindro (en español)
radio_interno = ", r_int, " # ", unidad, "
radio_externo = ", r_ext, " # ", unidad, "
altura = ", altura, " # ", unidad, "
diametro_externo = ", d_ext, " # ", unidad, "
grosor = ", grosor, " # ", unidad, " (valor calculado directamente en R)
unidad = '", unidad, "'

# Crear figura con tamaño reducido
fig, ax = plt.subplots(figsize=(4, 4)) # Reducido de 5x5 a 4x4

# Colores
azul_cilindro = '#4682B4' # Azul para contornos
azul_relleno = '#EBF5FF' # Azul muy claro para el relleno
gris_linea = '#777777' # Gris para líneas punteadas

# Factores de perspectiva para las elipses
factor_elipse = 0.35
vradio_ext = radio_externo * factor_elipse
vradio_int = radio_interno * factor_elipse

# Centros del cilindro
centro_superior = (0, altura)
centro_inferior = (0, 0)

# Rellenar el cilindro con color azul claro
# Crear coordenadas para el cuerpo exterior visible

```

```

angulos = np.linspace(-np.pi, 0, 50)
x_ext_frontal = radio_externo * np.cos(angulos)
y_ext_frontal_inf = vradio_ext * np.sin(angulos)
y_ext_frontal_sup = altura + vradio_ext * np.sin(angulos)

# Polígono para el cuerpo exterior
puntos_cuerpo_ext = []
for i in range(len(angulos)):
    puntos_cuerpo_ext.append((x_ext_frontal[i], y_ext_frontal_inf[i]))
puntos_cuerpo_ext.append((radio_externo, 0))
puntos_cuerpo_ext.append((radio_externo, altura))
for i in range(len(angulos)-1, -1, -1):
    puntos_cuerpo_ext.append((x_ext_frontal[i], y_ext_frontal_sup[i]))
poligono_cuerpo_ext = Polygon(puntos_cuerpo_ext, closed=True, facecolor=azul_relleno,
                              edgecolor='none', alpha=0.5, zorder=1)
ax.add_patch(poligono_cuerpo_ext)

# Polígono para el cuerpo interior (hueco)
x_int_frontal = radio_interno * np.cos(angulos)
y_int_frontal_inf = vradio_int * np.sin(angulos)
y_int_frontal_sup = altura + vradio_int * np.sin(angulos)

puntos_cuerpo_int = []
for i in range(len(angulos)):
    puntos_cuerpo_int.append((x_int_frontal[i], y_int_frontal_inf[i]))
puntos_cuerpo_int.append((radio_interno, 0))
puntos_cuerpo_int.append((radio_interno, altura))
for i in range(len(angulos)-1, -1, -1):
    puntos_cuerpo_int.append((x_int_frontal[i], y_int_frontal_sup[i]))
poligono_cuerpo_int = Polygon(puntos_cuerpo_int, closed=True, facecolor='white',
                              edgecolor='none', zorder=2)
ax.add_patch(poligono_cuerpo_int)

# Dibujar líneas verticales
ax.plot([radio_externo, radio_externo], [0, altura], color=azul_cilindro, linewidth=1.2, zorder=5)
ax.plot([-radio_externo, -radio_externo], [0, altura], color=azul_cilindro, linewidth=1.2, zorder=5)
ax.plot([radio_interno, radio_interno], [0, altura], color=azul_cilindro, linewidth=1.2, zorder=5)
ax.plot([-radio_interno, -radio_interno], [0, altura], color=azul_cilindro, linewidth=1.2, zorder=5)

# LÍNEAS PUNTEADAS PARA LAS PARTES NO VISIBLES DE LAS TAPAS SUPERIOR E INFERIOR
# Semiélipses posteriores (arco trasero)
angulos_traseros = np.linspace(0, np.pi, 100) # Más puntos para curvas más suaves
x_trasero_ext = radio_externo * np.cos(angulos_traseros)
y_trasero_ext_inf = vradio_ext * np.sin(angulos_traseros)
y_trasero_ext_sup = altura + vradio_ext * np.sin(angulos_traseros)
x_trasero_int = radio_interno * np.cos(angulos_traseros)
y_trasero_int_inf = vradio_int * np.sin(angulos_traseros)
y_trasero_int_sup = altura + vradio_int * np.sin(angulos_traseros)

# Dibujar líneas punteadas para las semiélipses posteriores
linea_punteada = {'linestyle': '--', 'color': gris_linea, 'linewidth': 1.0, 'zorder': 3,
                  'dashes': [3, 2]} # Líneas punteadas correctas

```

```

# Semiélipses posteriores inferiores
ax.plot(x_trasero_ext, y_trasero_ext_inf, **linea_punteada)
ax.plot(x_trasero_int, y_trasero_int_inf, **linea_punteada)

# Semiélipses posteriores superiores
ax.plot(x_trasero_ext, y_trasero_ext_sup, **linea_punteada)
ax.plot(x_trasero_int, y_trasero_int_sup, **linea_punteada)

# Dibujar semiélipses frontales visibles
# Semiélipses frontales inferiores
angulos_frontales = np.linspace(-np.pi, 0, 100)
x_frontal_ext = radio_externo * np.cos(angulos_frontales)
y_frontal_ext_inf = vradio_ext * np.sin(angulos_frontales)
x_frontal_int = radio_interno * np.cos(angulos_frontales)
y_frontal_int_inf = vradio_int * np.sin(angulos_frontales)

ax.plot(x_frontal_ext, y_frontal_ext_inf, color=azul_cilindro, linewidth=1.2, zorder=5)
ax.plot(x_frontal_int, y_frontal_int_inf, color=azul_cilindro, linewidth=1.2, zorder=5)

# Semiélipses frontales superiores
y_frontal_ext_sup = altura + vradio_ext * np.sin(angulos_frontales)
y_frontal_int_sup = altura + vradio_int * np.sin(angulos_frontales)
ax.plot(x_frontal_ext, y_frontal_ext_sup, color=azul_cilindro, linewidth=1.2, zorder=5)
ax.plot(x_frontal_int, y_frontal_int_sup, color=azul_cilindro, linewidth=1.2, zorder=5)

# Añadir puntos centrales
ax.plot(0, altura, 'o', color=azul_cilindro, markersize=3, zorder=6)
ax.plot(0, 0, 'o', color=azul_cilindro, markersize=3, zorder=6)

# Añadir flechas de radio
ax.add_patch(FancyArrowPatch((0, altura), (radio_interno, altura), arrowstyle='->',
                             color=azul_cilindro, linewidth=0.8, mutation_scale=8, zorder=6))
ax.add_patch(FancyArrowPatch((0, 0), (radio_externo, 0), arrowstyle='->',
                             color=azul_cilindro, linewidth=0.8, mutation_scale=8, zorder=6))

# FLECHAS Y ETIQUETAS DE DIMENSIONES
# Propiedades comunes para flechas
flecha_props = dict(arrowstyle='<->', color='black', linewidth=0.6, mutation_scale=6)

# 1. DIÁMETRO EXTERNO
diam_y = altura + vradio_ext*2 + altura*0.15
ax.add_patch(FancyArrowPatch((-radio_externo, diam_y), (radio_externo, diam_y), **flecha_props))
ax.text(0, diam_y + altura*0.06, f'Diámetro externo = {"", d_ext_fmt, "} {unidad}',
        fontweight='bold', ha='center', va='center', fontsize=7)

# 2. ALTURA
altura_x = radio_externo + radio_externo*0.3
ax.add_patch(FancyArrowPatch((altura_x, 0), (altura_x, altura), **flecha_props))
ax.text(altura_x + radio_externo*0.1, altura/2, 'Altura',
        fontweight='bold', rotation=90, ha='center', va='center', fontsize=7)

# 3. RADIO INTERNO
radio_int_label_pos = (radio_interno*2.0, altura + altura*0.2)

```

```

ax.plot([radio_interno, radio_int_label_pos[0]], [altura, radio_int_label_pos[1]], 'k-', linewidth=0.6)
ax.plot([radio_int_label_pos[0]-0.1, radio_int_label_pos[0]], [radio_int_label_pos[1], radio_int_label_pos[1]], 'k-', linewidth=0.6)
ax.text(radio_int_label_pos[0], radio_int_label_pos[1],
        f'Radio interno = {"", r_int_fmt, ""} {unidad}',
        fontweight='bold', ha='left', va='center', fontsize=7)

# 4. GROSOR
grosor_start = -radio_externo + (radio_externo - radio_interno)/2
ax.add_patch(FancyArrowPatch((-radio_interno, altura), (-radio_externo, altura), **flecha_props, zorder=2))
grosor_label_pos = (-radio_externo*2.0, altura + altura*0.2)
ax.plot([-radio_externo, grosor_label_pos[0]], [altura, grosor_label_pos[1]], 'k-', linewidth=0.6)
ax.plot([grosor_label_pos[0], grosor_label_pos[0]+0.1], [grosor_label_pos[1], grosor_label_pos[1]], 'k-', linewidth=0.6)
ax.text(grosor_label_pos[0], grosor_label_pos[1],
        f'Grosor = {"", grosor_fmt, ""} {unidad}',
        fontweight='bold', ha='right', va='center', fontsize=7)

# 5. RADIO EXTERNO
radio_ext_label_pos = (radio_externo*2.0, -altura*0.2)
ax.plot([radio_externo, radio_ext_label_pos[0]], [0, radio_ext_label_pos[1]], 'k-', linewidth=0.6)
ax.plot([radio_ext_label_pos[0]-0.1, radio_ext_label_pos[0]], [radio_ext_label_pos[1], radio_ext_label_pos[1]], 'k-', linewidth=0.6)
ax.text(radio_ext_label_pos[0], radio_ext_label_pos[1],
        f'Radio externo = {"", r_ext_fmt, ""} {unidad}',
        fontweight='bold', ha='left', va='center', fontsize=7)

# Ajustar límites y aspecto de la figura
margen = max(radio_externo, altura) * 0.7 # Reducido de 0.8 a 0.7
ax.set_xlim(-radio_externo - margen, radio_externo + margen)
ax.set_ylim(-margen*0.6, altura + margen) # Ajustado para reducir espacio vertical
ax.set_aspect('equal')
ax.axis('off')

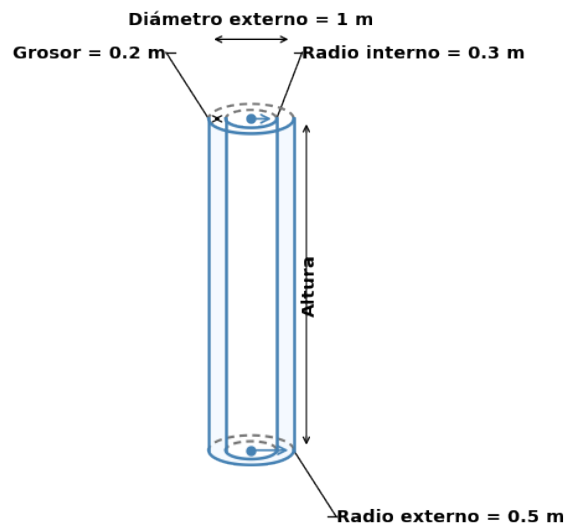
# Guardar la figura con espacio ajustado para etiquetas
plt.tight_layout(pad=0.8) # Reducido de 1.0 a 0.8
plt.savefig('cilindro_hueco.png', dpi=150, bbox_inches='tight', transparent=True) # Reducido DPI de 200 a 150
plt.close()
")

# Ejecutar código Python para generar la figura
py_run_string(codigo_python)

```

Question

José desea saber cuánto(a) agua se necesita para llenar un cilindro interno(a), pero solamente cuenta con las medidas de las dimensiones que muestra la figura.



Dimensión	Valor	Unidad
Radio interno	0,3	m
Radio externo	0,5	m
Diámetro externo	1	m
Grosor	0,2	m

¿Cuál medida le falta a José para hallar la cantidad deseada?

Answerlist

- Radio externo.
- Diámetro externo.
- Altura del cilindro.
- Perímetro del cilindro.

Solution

La respuesta correcta es: **Altura del cilindro.**

Para calcular el volumen de agua necesario para llenar el cilindro medio, necesitamos calcular el volumen de un cilindro, cuya fórmula es:

$$V = \pi \cdot r^2 \cdot h$$

Donde:

- r es el radio (en este caso, el radio interno = 0,3 m)
- h es la altura del cilindro

En el problema se nos proporciona:

- Diámetro externo = 1 m
- Radio interno = 0,3 m

Sin embargo, no se nos proporciona la altura del cilindro, que es esencial para calcular el volumen. Por lo tanto, a José le falta conocer la altura del cilindro para poder calcular la cantidad de agua necesaria.

Si tuviéramos la altura, el volumen se calcularía así:

$$V = \pi \cdot (0,3)^2 \cdot h = 0,28 \cdot h \text{ m}^3$$

Answerlist

- Falso
- Falso
- Verdadero
- Falso

Meta-information

exname: volumen_cilindro_hueco extype: schoice exsolution: 0010 exshuffle: TRUE exsection: Geometría|Volumen|Cilindro